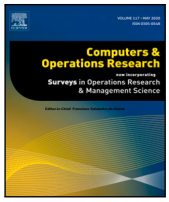
Computers & Operations Research 162 (2024) 106467

Contents lists available at ScienceDirect

Computers and Operations Research

journal homepage: www.elsevier.com/locate/cor



A unified exact approach for a broad class of vehicle routing problems with simultaneous pickup and delivery

Rafael Praxedes [a,b,*], Teobaldo Bulhões [c], Anand Subramanian [d], Eduardo Uchoa [e]

[a] Universidade Federal da Paraíba, Centro de Informática, Programa de Pós-graduação em Informática, Rua dos Escoteiros s/n, Mangabeira, 58055-000, João Pessoa, Brazil
[b] Università degli Studi di Parma, Dipartimento di Ingegneria e Architettura, Parco Area delle Scienze, 181/A - 43124, Parma, Italy
[c] Universidade Federal da Paraíba, Centro de Informática, Departamento de Computação Científica, Rua dos Escoteiros s/n, Mangabeira, 58055-000, João Pessoa, Brazil
[d] Universidade Federal da Paraíba, Centro de Informática, Departamento de Sistemas de Computação, Rua dos Escoteiros s/n, Mangabeira, 58055-000, João Pessoa, Brazil
[e] Universidade Federal Fluminense, Departamento de Engenharia de Produção, Rua Passo da Pátria 156, Campus Praia Vermelha, Bloco D, Sala 306, São Domingos, 24210-240, Niterói, Brazil

ARTICLE INFO

Keywords:
Routing
Pickup and delivery
Column generation
Cutting planes

ABSTRACT

The vehicle routing problem (VRP) with simultaneous pickup and delivery (VRPSPD) is a classical combinatorial optimization problem in which one aims at determining least-cost routes, starting and ending at the depot, while simultaneously meeting the pickup and delivery demands of geographically dispersed customers in a single visit, and without violating the capacity of the vehicle. In this work, we propose a unified branch-cut-and-price (BCP) algorithm to solve ten variants of the problem, considering not only the standard version but also those including additional attributes such as a heterogeneous fleet of vehicles, time windows, route duration, multiple depots, and location decisions. The resulting problem that generalizes all of these variants can be formulated as a heterogeneous location routing problem with simultaneous pickup and delivery and time windows (HLRPSPDTW), for which we propose a compact formulation to serve as the basis for an extended formulation associated with the BCP approach. The proposed exact method includes many of the features developed in recent years, such as a bucket graph-based labeling algorithm, rank-1 cuts with limited memory, and dynamic ng-sets. Extensive computational experiments were performed on more than 550 benchmark instances and our algorithm was capable of obtaining a number of new optimal solutions, as well as improved lower bounds, for all variants addressed in this paper.

1. Introduction

The *vehicle routing problem with simultaneous pickup and delivery* (VRPSPD) generalizes the *vehicle routing problem* (VRP) by allowing one to perform pickup and delivery services simultaneously, and it has been widely studied in the literature (Koç et al., 2020) since its seminal work (Min, 1989). Given a set of customers with pickup and delivery demands, the objective is to design a disjoint set of tours, starting and ending at a depot, so as to minimize the total travel costs while satisfying the capacity constraints of the vehicles.

The large amount of work related to VRPSPD can be justified not only by the challenge in solving the associated variants that might include different attributes such as heterogeneous fleet, route duration, time windows, multiple depots, and so on but also by its practical applications in the context of reverse logistics. In addition to the delivery of products, some companies have to manage the collection of material from the same customers to be reused or recycled, thus contributing to environmental sustainability. Examples arise in the distribution of beverages or liquefied petroleum gas, in which filled bottles or recipients filled with gas are delivered to the customers, and empty bottles or recipients are collected during the same visit.

As shown in Section 2, the vast majority of existing models and algorithms attempting to provide optimal solutions or lower bounds are focused on one or two VRPSPD variants. In this work, we are interested in the exact solution of a broad class of VRPSPDs capable of dominating most of such tailored methods. To this end, we propose an improved *branch-cut-and-price* (BCP) algorithm for the *heterogeneous location-routing problem with simultaneous pickup and delivery*

* Corresponding author at: Università degli Studi di Parma, Dipartimento di Ingegneria e Architettura, Parco Area delle Scienze, 181/A - 43124, Parma, Italy.
E-mail addresses: rafael.praxedes@unipr.it (R. Praxedes), tbulhoes@ci.ufpb.br (T. Bulhões), anand@ci.ufpb.br (A. Subramanian), uchoa@producao.uff.br (E. Uchoa).

https://doi.org/10.1016/j.cor.2023.106467
Received 21 March 2023; Received in revised form 24 September 2023; Accepted 29 October 2023
Available online 31 October 2023

Table 1
PDP surveys.

Survey	Topic
Berbeglia et al. (2007)	static PDP
Parragh et al. (2008a)	PDP (linehauls and backhauls)
Parragh et al. (2008b)	PDP (other problems)
Berbeglia et al. (2010)	dynamic PDP
Battarra et al. (2014)	PDP for goods transportation
Koç et al. (2020)	VRPSPD
Bouanane et al. (2022)	VRPSPD

and with time windows (HLRPSPDTW) considering symmetric or asymmetric costs, which can be seen as a generic VRPSPD variant comprising most of the usual VRP attributes.

The HLRPSPDTW, which is clearly $\mathcal{NP}$-hard, includes a number of existing problems as special cases, and we consider the following ten variants: (i) VRPSPD with symmetric costs; (ii) *asymmetric* VRPSPD (AVRPSPD); (iii) VRP *with mixed pickup and delivery* (VRPMPD), in which each customer has either pickup or delivery demand; (iv) VRPSPD *with time limit* (VRPSPDTL) (on route duration); (v) VRPMPD *with time limit* (VRPMPDTL); (vi) VRPSPD *with time windows* (VRPSPDTW); (vii) *flexible delivery and pickup problem with time windows* (FDPPTW), which can be seen as a VRPSPD with distinct time windows for each type of demand; (viii) *heterogeneous* VRPSPD (HVRPSPD), in which the vehicles might have distinct capacities; (ix) *multi-depot* VRPMPD (MDVRPMPD); and (x) *location-routing* VRPSPD (LRPSPD).

In contrast to the BCP put forward in Subramanian et al. (2013), our approach addresses the delivery and pickup resources independently in the pricing subproblem, and their simultaneity is ensured via lazy cuts. Also, it includes many of the features developed in recent years, such as a bucket graph-based labeling algorithm, rank-1 cuts with limited memory, and dynamic ng-sets. The proposed BCP algorithm was implemented using the VRPSolver framework (Pessoa et al., 2020). It was tested on more than 530 benchmark instances with up to 400 customers, and it was found capable of improving the lower bound (LB) values of 58.0% instances and proving the optimality of 44.3% open problems.

The remainder of the paper is organized as follows. Section 2 briefly reviews the related work. Section 3 formally defines the HLRPSPDTW, whereas Section 4 presents a compact mathematical formulation for the problem and how it can be adapted to address the ten variants considered in this work. Section 5 explains the proposed exact algorithm. Section 6 reports the results obtained after extensive computational experiments. Finally, Section 7 presents the concluding remarks.

2. Related work

Many surveys have addressed *pickup and delivery* problems (PDPs) in the last 15 years and they are listed in Table 1. The recent ones by Bouanane et al. (2022), Koç et al. (2020) specifically approached the VRPSPDs, while those by Parragh et al. (2008a), Battarra et al. (2014) include a brief review of the problem.

Berbeglia et al. (2007) classified the VRPSPD as a *one-to-many-to-one* PDP, meaning that the entire amount to be delivered and collected comes from and goes to the depot, respectively. Note that customers do not serve as suppliers to one another as it happens in *many-to-many* PDPs like those arising in *bike sharing rebalancing problems*. Moreover, *one-to-one* PDPs, as those reviewed in Parragh et al. (2008b), are mostly related to the transportation of people from one location to the other like in *dial-a-ride problems*, and the vehicles start/return empty from/to the depot, in contrast to the VRPSPD.

Table 2 compiles all benchmark instances used in the ten variants considered in this work, whereas Table 3 summarizes the main exact approaches proposed for them. The former describes the authors of each set of instances, from where they were derived, their size (#*inst*), the percentage of instances solved to optimality (%*opt*), as well as the number of customers (n), depots (m) and types of vehicles ($\#K$) (note that $\#K = 1$ implies that there is only one type of vehicle and thus the fleet is homogeneous). The latter contains the variants addressed in each work, the specific exact method used to solve the problem, the benchmark considered and the number of instances from such dataset, the number of optimal solutions found (#*opt*), the larger instance solved to optimality (n_{max}), and the smaller instance unsolved to optimality (n_{min}). In this work, we consider the most challenging benchmark datasets, i.e., those with open instances and which were made available by the authors who proposed them.

From Tables 2 and 3, it can be observed that the classic (and symmetric) version of the problem (VRPSPD) is the one that received the most attention from the literature, as opposed to its asymmetric counterpart (AVRPSPD). Nevertheless, many VRPSPD instances ranging from 75 to 400 customers remain open. The best exact algorithms for the problem are the branch-and-cut (BC) and BCP proposed in Subramanian et al. (2011, 2013), respectively. The referred methods were also employed by the same authors to solve the VRPMPD, while the former was used in Subramanian (2012) for solving the MDVRPMDP. Moreover, to our knowledge, the variants that include route duration constraints, i.e., VRPSPDTL and VRPMPDTL, have never been approached by exact algorithms. In the present work, we provide the first dual bounds for these latter problems and we also report improved bounds for many instances of the former ones.

Regarding the variants with time windows, i.e., VRSPDTW and FDPPTW, there are relatively small-size instances (containing 25 and 10 customers, respectively) still unsolved. The same observation can be made for the variant considering a heterogeneous fleet of vehicles (HVRPSPD) and the one involving location decisions (LRPSPD). They have mostly been exactly addressed only by mathematical formulations that were solved using a commercial solver. Our proposed exact algorithm substantially improves the existing bounds for such problems as reported in Section 6.

3. Problem description

Let $V_C = \{1, \dots, n\}$, $V_D = \{n+1, \dots, n+m\}$, and $V'_D = \{(n+1)+m, \dots, (n+m)+m\}$ be the set of n customers, m depots, and m depot copies, respectively. It is important to mention that the depots $n+d \in V_D$ and $(n+d)+m \in V'_D$, $d \in \{1, \dots, m\}$, are identical. Concerning the arcs, let $A_d = \{(i,j) : i \in V_C \cup \{n+d\}, j \in V_C \cup \{n+d+m\}, i \neq j\}, d \in \{1, \dots, m\}$, be the arc set connecting the customers to one another and to the depot $n+d \in V_D$ or its correspondent $(n+d)+m \in V'_D$. Hence, one can define the directed graph $G = (V, A)$, with $V = V_C \cup V_D \cup V'_D$ and $A = \cup_{d=1}^{m} A_d$. For each arc $a = (i,j) \in A$, c_a and t_a denote the cost and travel time, respectively, where the latter includes the service time of customer i. In addition, let K be the set of all vehicle types available. The vehicles of type $k \in K$ have a variable cost α_k associated with each distance unit traveled and a fixed cost β_k related to their usage, whereas $\gamma_d, d \in \{1, \dots, m\}$, represents the fixed cost of depot $n+d \in V_D$. Moreover, define $Q_k, k \in K$, and $Q'_d, d \in \{1, \dots, m\}$, as the capacities of each type of vehicle and of the *d-th* depot, respectively, U_k the maximum number of vehicles for each type and U'_d the same limit, although for each depot. Each customer $j \in V_C$ has non-negative pickup p_j and delivery q_j demands, as well as time windows $[e_j, l_j]$. The demands of the nodes representing the depots $j \in V_D \cup V'_D$ are zero, and they have time windows $[e_j, l_j] = [0, T_{max}]$, where T_{max} is the maximum route duration.

The problem consists in determining which depots will be used, which customers will be assigned to each of the selected depots, and the least-cost routes while meeting the pickup and delivery demands of all customers and satisfying the following constraints: (i) each customer must be visited exactly once; (ii) the same vehicle can be used in at most one route; (iii) each route must start and end at the same depot; (iv) the time windows of each customer must be satisfied; (v) the route

Table 2
Summary of all benchmark instances.

Variant	Set	Proposed by	Derived from	#inst	n	m	#K	%opt
VRPSPD	SN	Salhi and Nagy (1999)	–	14	50 – 200	1	1	29
	D	Dethloff (2001)	–	40	50	1	1	100
	MG	Montané and Galvão (2006)	–	18	100 – 400	1	1	44
	DC1	Dell'Amico et al. (2006)	Solomon (1987)	12	20 – 40	1	1	100
	DC2C	Dell'Amico et al. (2006)	Toth and Vigo (1997)	18	20 – 40	1	1	100
	DC3C	Dell'Amico et al. (2006)	VRPLIB	36	20 – 40	1	1	50
	M05	Mitra (2005)	–	8	19	1	1	100
	CW06	Chen and Wu (2006)	Gehring and Homberger (2002)	25	15 – 20	1	1	100
	AV21D	Agarwal and Venkateshan (2021)	–	10	50	1	1	100
	AV21C	Agarwal and Venkateshan (2021)	–	10	50	1	1	80
VRPMPD	SN	Salhi and Nagy (1999)	–	21	50 – 199	1	1	52
VRPSPDTL	SN	Salhi and Nagy (1999)	–	14	50 – 199	1	1	0
VRPMPDTL	SN	Salhi and Nagy (1999)	–	21	50 – 199	1	1	0
VRPSPTW	W12	Wang and Chen (2012)	Solomon (1987)	65	10 – 100	1	1	17
	AM02	Angelelli and Mansini (2002)	Solomon (1987)	29	20	1	1	100
HVRPSPD	Avc1	Avci and Topaloglu (2016)	–	14	10 – 100	1	2 – 4	0
	QB1	Qu and Bard (2015)	Real-world data	24	7 – 50	1	4	79
	QB2	Qu and Bard (2015)	–	20	20 – 50	1	4	95
MDVRPMPD	SN	Salhi and Nagy (1999)	–	33	50 – 249	2 – 5	1	12
LRPSPD	Bar11	Karaoglan et al. (2011)	Barreto et al. (2007)	60	8 – 100	2 – 15	1	62
	Prod11	Karaoglan et al. (2011)	Prins et al. (2004)	88	20 – 100	5 – 10	1	20
	Prod12	Karaoglan et al. (2012)	Prins et al. (2004)	360	10 – 100	3 – 10	1	16
AVRPSPD	RZC1	Rieck and Zimmermann (2013)	Real-world data	100	20 – 60	1	1	90
	RZC6	Rieck and Zimmermann (2013)	Dethloff (2001)	40	50	1	1	50
	RZC7	Rieck and Zimmermann (2013)	Salhi and Nagy (1999)	2	50	1	1	100
	AV20R	Agarwal and Venkateshan (2020)	–	10	45	1	1	100
	AV20D	Agarwal and Venkateshan (2020)	–	10	35	1	1	80
FDPPTW	W13	Wang and Chen (2013)	Wang and Chen (2012)	68	5 – 100	1	1	6

duration must not be exceeded; (vi) the number of routes must respect the maximum number of vehicles of a certain type, as well the number of vehicles available at each depot; and (vii) the capacity of each depot must not be exceeded for both kinds of demand (pickup and delivery) separately. Furthermore, given a route $R = (v_0^R, v_1^R, v_2^R, \dots, v_{r-1}^R, v_r^R)$, where v_0^R and v_r^R represent the same depot, let Q_R be the capacity of the vehicle associated with the route R. The following capacity constraints must be satisfied: (viii) $\sum_{i=1}^{r} q_{v_i^R} \le Q_R$; (ix) $\sum_{i=1}^{r} p_{v_i^R} \le Q_R$; and (x) $\sum_{i=1}^{(\kappa-1)} p_{v_i^R} + \sum_{i=\kappa}^{r} q_{v_i^R} \le Q_R, \kappa \in \{2, \dots, r\}$.

4. Mathematical formulations

Let $x_a^{kd}, k \in K, d \in \{1, \dots, m\}, a \in A_d$, be a non-negative integer variable that computes the number of times arc a was traversed by a vehicle of type k that started its route at depot $n+d$, and let binary variable $y_d, d \in \{1, \dots, m\}$, assume the value 1 if the depot $n+d$ has been used. In addition, continuous variables P_a and D_a, $a \in A$, determine the pickup and delivery loads on arc $a = (i, j)$, respectively, where the former corresponds to the total amount collected up to customer i, while the latter indicates the total amount delivered in all customers that follow customer i, starting from customer j. Furthermore, let $T_j, j \in V$, be a continuous variable representing the service start time at vertex j. For $d \in \{1, \dots, m\}$ and $j \in V$, we also define $\delta_d^+(j) = \{(\bar{i}, \bar{j}) \in A_d : \bar{i} = j\}$ and $\delta_d^-(j) = \{(\bar{i}, \bar{j}) \in A_d : \bar{j} = j\}$, and analogously $\delta^+(j) = \cup_{d=1}^m \delta_d^+(j)$ and $\delta^-(j) = \cup_{d=1}^m \delta_d^-(j)$. Finally, we define $V_D(a)$ as the set of depot indices associated with arc $a = (i, j) \in A$, depending if i or j belongs or not to sets V_D and V_D', respectively. A formal description of $V_D(a)$, where $a = (i, j)$, is presented in Eq. (1).

$$V_D(a) = \begin{cases} \{1, \dots, m\}, & \text{if } i \notin V_D, j \notin V_D' \\ \{i - n\}, & \text{if } i \in V_D \\ \{j - n - m\}, & \text{if } j \in V_D' \end{cases} \quad (1)$$

We can write a compact mathematical formulation for the problem as follows:

$$\text{(F1)} \quad \min \sum_{k \in K} \sum_{d=1}^{m} \sum_{a \in \delta_d^+(n+d)} \beta_k x_a^{kd} + \sum_{d=1}^{m} \gamma_d y_d + \sum_{k \in K} \sum_{d=1}^{m} \sum_{a \in A_d} \alpha_k c_a x_a^{kd} \quad (2)$$

s.t.

$$\sum_{k \in K} \sum_{d=1}^{m} \sum_{a \in \delta_d^-(j)} x_a^{kd} = 1 \qquad j \in V_C \quad (3)$$

$$\sum_{a \in \delta_d^+(i)} x_a^{kd} = \sum_{a \in \delta_d^-(i)} x_a^{kd} \qquad k \in K, d \in \{1, \dots, m\}, i \in V_C \quad (4)$$

$$\sum_{a \in \delta^-(j)} D_a - \sum_{a \in \delta^+(j)} D_a = q_j \qquad j \in V_C \quad (5)$$

$$\sum_{a \in \delta^+(j)} P_a - \sum_{a \in \delta^-(j)} P_a = p_j \qquad j \in V_C \quad (6)$$

$$D_a + P_a \le \sum_{k \in K} \sum_{d \in V_D(a)} Q_k x_a^{kd} \qquad a \in A \quad (7)$$

$$\sum_{d=1}^{m} \sum_{a \in \delta_d^+(n+d)} x_a^{kd} \le U_k \qquad k \in K \quad (8)$$

$$\sum_{k \in K} \sum_{a \in \delta_d^+(n+d)} x_a^{kd} \le U_d' \qquad d \in \{1, \dots, m\} \quad (9)$$

$$\sum_{k \in K} \sum_{a=(i,j) \in A_d} q_i x_a^{kd} \le Q_d' y_d \qquad d \in \{1, \dots, m\} \quad (10)$$

$$\sum_{k \in K} \sum_{a=(i,j) \in A_d} p_i x_a^{kd} \le Q_d' y_d \qquad d \in \{1, \dots, m\} \quad (11)$$

$$T_j \ge T_i + t_a - Z\left(1 - \sum_{k \in K} \sum_{d \in V_D(a)} x_a^{kd}\right) \qquad a = (i, j) \in A \quad (12)$$

Table 3
Summary of exact approaches proposed for the VRPSPD and its main variants.

Variant	Authors (year)	Method	Considered sets	#inst	#opt	n_{max}	n_{min}
VRPSPD	Dell'Amico et al. (2006)	Branch-and-Price (BP)	DC1	12	9	40	40
			DC2C	18	18	40	–
			DC3C	18	16	40	40
	Subramanian et al. (2010)	Flow Formulations	SN	14	2	50	75
			D	40	17	50	50
			MG	12	3	100	100
	Subramanian et al. (2011)	BC with Lazy Separation	SN	14	4	100	75
			D	40	40	50	–
			MG	12	5	200	100
	Subramanian et al. (2013)	BCP	SN	14	–	–	–
			D	40	20	50	50
			MG	12	3	100	100
	Rieck and Zimmermann (2013)	Flow and Commodity Formulations	M05	8	8	19	–
			CW06	25	25	20	–
			DC1	12	12	40	–
			DC3C	18	18	40	–
	Agarwal and Venkateshan (2021)	Formulation with no-good cuts	D	19	19	50	–
			AV21D	10	9	50	50
			AV21C	10	10	50	–
VRPMPD	Subramanian et al. (2011)	BC with Lazy Separation	SN	21	10	120	75
	Subramanian et al. (2013)	BCP	SN	21	1	75	50
VRPSPDTW	Angelelli and Mansini (2002)	BP	AM02	29	29	20	–
	Wang and Chen (2012)	Formulation	W12	9	5	50	25
	Agius et al. (2022)[a]	BP	W12	65	11	100	25
FDPPTW	Wang and Chen (2013)	Formulation	W13	12	4	10	10
HVRPSPD	Qu and Bard (2015)[a]	BCP	QB1	24	19	38	29
			QB2	20	19	50	50
	Avci and Topaloglu (2016)	Formulation	Avc1	14	0	–	10
MDVRPMPD	Subramanian (2012)	BC with Lazy Separation	SN	21	4	100	50
LRPSPD	Karaoglan et al. (2011)	BC with SA	Bar11	60	37	88	50
			Prod11	88	18	50	50
	Karaoglan et al. (2012)	Flow and Node Formulations	Prod12	96	58	30	15
AVRPSPD	Rieck and Zimmermann (2013)	Flow and Commodity Formulations	RZC1	100	90	60	30
			RZC6	40	20	50	50
			RZC7	2	2	50	–
	Agarwal and Venkateshan (2020)	Formulation with no-good cuts	RCZ6	20	20	50	–
			AV20R	10	10	45	–
			AV20D	10	8	35	35

[a] The problem addressed by the authors does not exactly correspond to the respective variant.

$$e_j \le T_j \le l_j \qquad j \in V \tag{13}$$

$$D_a, P_a \ge 0 \qquad a \in A \tag{14}$$

$$x_a^{kd} \in \mathbb{Z}_+ \qquad k \in K, d \in \{1, \dots, m\}, a \in A_d \tag{15}$$

$$y_d \in \{0, 1\} \qquad d \in \{1, \dots, m\}. \tag{16}$$

The objective function (2) minimizes the total cost, including travel costs and the variable and fixed costs associated with the vehicles and depots. Constraints (3) and (4) impose that each customer must be visited exactly once and by the same vehicle. Constraints (5) and (6) ensure the flow conservation associated with the delivery and pickup demands, respectively. Constraints (7) guarantee that the simultaneous pickup and delivery load will not exceed the vehicle's capacity at any point of the route. Constraints (8) and (9) imply that the limit of vehicles number for each type of fleet or depot must be respected, respectively. Constraints (10) state that the sum of the delivery demands of the customers associated with the *d-th* depot must not be exceed its capacity. Constraints (11) have a similar meaning, but for pickup demands. Constraints (12) determine the service start times, where Z is a sufficiently large constant. Constraints (13) ensure that the customer and depot time windows are respected. Finally, constraints (14)–(16) define the domain of the variables. Table 4 shows how each of the ten variants considered can be formulated as a particular case of the generic model given by (2)–(16). In particular, we specify, for each variant, how the input data can be adapted and which constraints should be relaxed in order to represent such case.

Note that constraints (4)–(7) and (12)–(15) define sets of paths in G satisfying the capacity and time window constraints, but not ensuring that each customer is visited once. These paths were denoted as *pd*-routes by Subramanian et al. (2013). Therefore, let $\mathcal{P}_d^k$ be the set of *pd*-routes in G associated with the vehicle type $k \in K$ and the depot $d \in \{1, \dots, m\}$, that is, $\mathcal{P}_d^k$ contains the elementary paths in G that: (i)

Table 4
Differences among the variants considered.

Variant	Customers	Vehicle types	Depots	Arcs	Constraints relaxed
VRPSPD	$p_j \geq 0, q_j \geq 0,\ e_j = 0, l_j = \infty$	$Q_k = Q,\ \alpha_k = 1,\ \beta_k = 0,\ U_k < \infty$	$Q'_d = \infty,\ \gamma_d = 0,\ U'_d = \infty$	$c_a = c_{a'}$	(10), (11), (12) (13), (16)
VRPMPD	$p_j > 0 \Rightarrow q_j = 0,\ p_j = 0 \Rightarrow q_j > 0$ $e_j = 0, l_j = \infty$				
VRPSPDTL	$p_j \geq 0,\ q_j \geq 0,\ e_j = 0, l_j = T_{max}$				(10), (11), (16)
VRPMPDTL	$p_j > 0 \Rightarrow q_j = 0,\ p_j = 0 \Rightarrow q_j > 0$ $e_j = 0, l_j = T_{max}$				
VRPSPDTW	$p_j \geq 0,\ q_j \geq 0,\ e_j \geq 0, l_j > 0$				
FDPPTW	$p_j > 0 \Rightarrow q_j = 0,\ p_j = 0 \Rightarrow q_j > 0$ $e_j \geq 0, l_j > 0$				
HVRPSPD	$p_j \geq 0, q_j \geq 0,\ e_j = 0, l_j = \infty$	$Q_k > 0,\ \alpha_k > 0,\ \beta_k > 0,\ U_k = \infty$			(10), (11), (12) (13), (16)
MDVRPMPD	$p_j > 0 \Rightarrow q_j = 0,\ p_j = 0 \Rightarrow q_j > 0$ $e_j = 0, l_j = \infty$	$Q_k = Q,\ \alpha_k = 1,\ \beta_k = 0,\ U_k = \infty$	$Q'_d = \infty,\ \gamma_d = 0,\ U'_d < \infty$		(12), (13)
LRPSPD	$p_j \geq 0,\ q_j \geq 0,\ e_j = 0, l_j = \infty$		$Q'_d > 0,\ \gamma_d > 0,\ U'_d < \infty$		
AVRPSPD		$Q_k = Q,\ \alpha_k = 1,\ \beta_k = 0,\ U_k < \infty$	$Q'_d = \infty,\ \gamma_d = 0,\ U'_d = \infty$	$c_a \neq c_{a'}$	(10), (11), (12) (13), (16)

$j \in V_C,\ k \in K,\ d \in \{1, \ldots, m\},\ a = (i, j) \in A,$ and $a' = (j, i) \in A$.

start at vertex $n+d \in V_D$; (ii) end at vertex $(n+m)+d \in V'_D$; (iii) can be traversed by a vehicle of type k while satisfying its capacity and also the customer's time windows.

Formulation F1 can be strengthened by including variables associated with the pd-routes. For a path $P \in \mathcal{P}_d^k$, let λ_P be a variable indicating whether P is used in the solution and let h_a^P be the number of times that arc $a \in A_d$ appears in it. The resulting extended formulation is:

(Ext-F1) min (2)

s.t.

(3)–(16)

$$x_a^{kd} = \sum_{P \in \mathcal{P}_d^k} h_a^P \lambda_P \qquad k \in K, d \in \{1, \ldots, m\}, a \in A_d \tag{17}$$

$$\lambda_P \in \mathbb{Z}_+ \qquad k \in K, d \in \{1, \ldots, m\}, P \in |\mathcal{P}_d^k. \tag{18}$$

Constraints (17)–(18) force variables x to represent the sum of pd-routes. Those constraints make constraints (4)–(7) and (12)–(15) redundant, so they can be removed. Note that bounding λ and $\boldsymbol{x}$ variables from above is unnecessary due to constraints (3). By also substituting every x variable by their equivalent in terms of the λ variables, given by (17), we can write the formulation defined by (19)–(26).

$$\min \sum_{k \in K} \sum_{d=1}^{m} \sum_{a \in \delta_d^+(n+d)} \sum_{P \in \mathcal{P}_d^k} (\beta_k h_a^P) \lambda_P + \sum_{d=1}^{m} \gamma_d y_d + \sum_{k \in K} \sum_{d=1}^{m} \sum_{a \in A_d} \sum_{P \in \mathcal{P}_d^k} (\alpha_k c_a h_a^P) \lambda_P \tag{19}$$

s.t.

$$\sum_{k \in K} \sum_{d=1}^{m} \sum_{a \in \delta_d^-(j)} \sum_{P \in \mathcal{P}_d^k} h_a^P \lambda_P = 1 \qquad j \in V_C \tag{20}$$

$$\sum_{d=1}^{m} \sum_{a \in \delta_d^+(n+d)} \sum_{P \in \mathcal{P}_d^k} h_a^P \lambda_P \leq U_k \qquad k \in K \tag{21}$$

$$\sum_{k \in K} \sum_{a \in \delta_d^+(n+d)} \sum_{P \in \mathcal{P}_d^k} h_a^P \lambda_P \leq U'_d \qquad d \in \{1, \ldots, m\} \tag{22}$$

$$\sum_{k \in K} \sum_{a=(i,j) \in A_d} \sum_{P \in \mathcal{P}_d^k} (q_i h_a^P) \lambda_P \leq Q'_d y_d \qquad d \in \{1, \ldots, m\} \tag{23}$$

$$\sum_{k \in K} \sum_{a=(i,j) \in A_d} \sum_{P \in \mathcal{P}_d^k} (p_i h_a^P) \lambda_P \leq Q'_d y_d \qquad d \in \{1, \ldots, m\} \tag{24}$$

$$\lambda_P \in \mathbb{Z}_+ \qquad k \in K, d \in \{1, \ldots, m\}, P \in \mathcal{P}_d^k \tag{25}$$

$$y_d \in \{0, 1\} \qquad d \in \{1, \ldots, m\}. \tag{26}$$

As this formulation has an exponential number of variables, it needs to be solved using column generation. The BCP algorithm described in the following section works over a formulation very similar to the one described above. The difference is that, in order to make the pricing subproblems more tractable, the definition of the paths associated with the variables is relaxed, so they do not necessarily satisfy the capacity constraints in the middle of the route. In other words, for a route corresponding to a vehicle type $k \in K$, the total pickup and the total delivery should be less or equal to Q_K. However, one does not check in the middle of the route whether the sum of the already performed pickups plus the sum of the deliveries yet to be performed is less or equal to Q_K. Moreover, the route elementarity is also relaxed, so, it is possible to have cycles. Actually, we adopt the ng-path relaxation proposed by Baldacci et al. (2011), which is the state-of-the-art mechanism that offers the best compromise between linear relaxation strength and pricing time. More precisely, collection $\mathcal{M}$ contains a memory $M_j \subseteq V_C$ for each vertex $j \in V$, and thus a path P is ng-feasible iff between two visits to a customer j there exists a vertex i such that $j \notin M_i$. In other words, a cycle on a customer j is allowed iff the last visit to j is "forgotten" before the next visit to j. Hence, the BCP works with the sets $\hat{\mathcal{P}}_d^k(\mathcal{M}) \supseteq \mathcal{P}_d^k$ also containing those relaxed paths. The proposed algorithm is able to adjust the collection $\mathcal{M}$ dynamically, as described in Section 5.

We note that routes with cycles may appear in fractional solutions but are naturally avoided in integer solutions. This happens because all those routes have a coefficient greater or equal to two in Constraints (20). On the other hand, a path $P \in \hat{\mathcal{P}}_d^k$ violating the capacity in the middle of the route that appears in an integer solution is removed by the following constraint:

$$\sum_{a \in A_d} h_a^P x_a^{kd} \leq \sum_{a \in A_d} h_a^P - 1, \tag{27}$$

Hereafter, we denote as F2($\mathcal{M}$) the formulation defined by (19)–(26), but considering the paths of the set $\hat{\mathcal{P}}_d^k(\mathcal{M})$ instead of those in $\mathcal{P}_d^k$ and also including the no-good cuts in format (27).

5. Branch-cut-and-price approach

This section describes the main components of the BCP approach, in particular, the algorithm used to solve the pricing problem, the procedure employed to solve the master problem, as well as the cuts and branching strategies.

Algorithm 1 Computing the coefficient of a path $P = (v_0^P, v_1^P, \dots, v_{r-1}^P, v_r^P)$ in the R1C induced by multipliers μ_j, $j \in V_C$. Left: with full memory. Right: with a vertex-based memory N.

```
function coeff(P, μ)
state ← 0, coeff ← 0
for i ← 1 to r − 1 do
    j ← v_i^P
    state ← state + μ_j
    if state ≥ 1 then
        state ← state − 1
        coeff ← coeff + 1
return coeff
```

```
function coeff(P, μ, N)
state ← 0, coeff ← 0
for i ← 1 to r − 1 do
    j ← v_i^P
    if j ∉ N then
        state ← 0
    state ← state + μ_j
    if state ≥ 1 then
        state ← state − 1
        coeff ← coeff + 1
return coeff
```

5.1. Cuts

The first family of cuts is obtained by a Chvátal–Gomory rounding of the set-partitioning constraints. One defines a multiplier $u_j \in \mathbb{R}_+$ for each constraint in (20) and rounds down the coefficients in the resulting linear combination of the constraints to obtain the so-called *Rank-1 Chvátal–Gomory cut* (R1C) in (28).

$$\sum_{d=1}^{m} \sum_{k \in K} \sum_{P \in \mathcal{P}_d^k} \left(\left\lfloor \sum_{j \in V_c} u_j \sum_{a \in \delta_d^-(j)} h_a^P \right\rfloor \right) \lambda_P \leq \left\lfloor \sum_{j \in V_C} u_j \right\rfloor \tag{28}$$

It is known that it suffices to consider rational multipliers $u_j \in [0, 1]$ to generate all non-dominated Chvátal–Gomory cuts (Caprara and Fischetti, 1996). However, when the procedure is applied to set-partitioning constraints (like Constraints (20)), Pecin et al. (2017c) could determine the precise sets of rational multipliers with up to five non-zeros that generate non-dominated R1Cs. For three non-zeros, there is a single such set; for four non-zeros, there are two sets (not counting symmetries) and, for five non-zeros, there are five sets (not counting symmetries). Those are the multipliers used in this work.

According to the classification by Poggi and Uchoa (2003), R1Cs are *non-robust* as they change the structure of the pricing subproblem. Therefore, one should separate them in a controlled way to avoid unreasonable computational times to solve the pricing subproblem. Despite that, such cuts are widely adopted in current exact methods for VRPs (Jepsen et al., 2008; Baldacci et al., 2011; Contardo and Martinelli, 2014; Pecin et al., 2017b).

Pecin et al. (2017b) introduced a breakthrough mechanism that allows for better management of R1Cs. Let $B = \{j \in V_C : \mu_j > 0\}$ be the subset of customers with positive multiplier in a R1C. The idea is to define a memory N, $B \subseteq N \subseteq V_C$, for each cut so that the smaller the memory, the smaller the cut's impact in the pricing subproblem. The *limited memory R1C* (lm-R1C) is a weakening of the original R1C, as the coefficient of a path P in the former is less than or equal to the coefficient in the latter (see Algorithm 1, function in the left would compute the original coefficients in (28), the function in the right indicates how a vertex-based memory changes those coefficients). However, the memories can be dynamically adjusted based on the fractional solutions and eventually both versions of the cut converge to the same bounds (Pecin et al., 2017b). As demonstrated by Pecin et al. (2017a), in some cases, it is beneficial to define the cut memory as a set of arcs instead of a set of vertices. Our algorithm can be set to use either vertex memories or arc memories. The detailed configuration for each variant will be given in the appendix.

The second family of cuts consists of the well-known *rounded capacity inequalities* (RCIs) introduced by Laporte and Nobert (1983) for the Capacitated VRP (CVRP). The RCIs for pickup and for delivery demands are presented in (29) and (30), respectively. Up to 100 RCIs are separated in every cut round using the CVRPSEP package (Lysgaard et al., 2004). In contrast to R1Cs, RCIs are classified as *robust* since the values of the corresponding dual variables translate into reduced costs without changing the pricing subproblem structure.

$$\sum_{k \in K} \sum_{d=1}^{m} \sum_{a \in \delta_d^-(S)} x_a^{kd} \geq \left\lceil \frac{\sum_{i \in S} p_i}{\max_{k \in K} Q_k} \right\rceil \qquad S \subseteq V_C \tag{29}$$

$$\sum_{k \in K} \sum_{d=1}^{m} \sum_{a \in \delta_d^-(S)} x_a^{kd} \geq \left\lceil \frac{\sum_{i \in S} q_i}{\max_{k \in K} Q_k} \right\rceil \qquad S \subseteq V_C \tag{30}$$

5.2. Pricing subproblem

In this section, we briefly describe the pricing algorithm for a vehicle type $k \in K$ and depot index $d \in \{1, \dots, m\}$. Let ξ_j be the value of the dual variable of constraint (20) for each $j \in V_C$, and ν_k be the value of the dual variable of constraint (21) for vehicle type k. For ease of presentation, define $\xi_j = 0$ for a vertex $j \in V_D \cup V_D'$. Furthermore, define σ_d^1, σ_d^2 and σ_d^3, respectively, as the values of the dual variables of constraints (22), (23) and (24) regarding depot index d. Define n^{RCI} as the number of active RCIs, and let ω_l and h'_{al} denote, respectively, the value of the dual variable and the coefficient of an arc $a \in A$ in lth active RCI. Let n^{R1C} denote the number of active R1Cs. The memory, vector of multipliers, and value of the dual variable associated with the lth active R1C are denoted as N^l, $\boldsymbol{\mu}^l$, and π^l, respectively. The coefficient of a path P in the lth R1C is denoted as $\texttt{coeff}(P, \mu^l, N^l)$. Therefore, the contribution of an arc $a = (i, j) \in A_d$ to the reduced cost of a path is given by Eq. (31), and the reduced cost of a path $P \in \hat{\mathcal{P}}_d^k(\mathcal{M})$ is defined by Eq. (32).

$$\bar{c}_a = \begin{cases} \alpha_k c_a + \beta_k - \xi_j - \nu_k - \sigma_d^1 \\ \quad -q_i \sigma_d^2 - p_i \sigma_d^3 - \sum_{l=1}^{n^{\text{RCI}}} h'_{al} \omega_l & \text{if } a \in \delta_d^+(n+d) \\ \alpha_k c_a - \xi_j - q_i \sigma_d^2 - p_i \sigma_d^3 - \sum_{l=1}^{n^{\text{RCI}}} h'_{al} \omega_l & \text{otherwise} \end{cases} \tag{31}$$

$$\bar{c}_P = \sum_{a \in A_d} h_a^P \bar{c}_a - \sum_{l=1}^{n^{\text{R1C}}} \texttt{coeff}(P, \mu^l, N^l) \pi^l \tag{32}$$

Note that the dual variables of the robust constraints and cuts only appear in (31), while the dual variables of the non-robust cuts appear as additional terms in (32).

The pricing subproblem consists in finding a path $P \in \hat{\mathcal{P}}_d^k(\mathcal{M})$ with negative reduced cost or proving that there is no such a path. As usual in the VRP literature, we solve it by means of a labeling algorithm, whose basic principle is to enumerate paths with the aid of dominance rules to eliminate unpromising partial paths. The labeling approach can be implemented in several ways, especially when it comes to the data structures that keep the labels representing partial paths. We adopt the *bucket graph-based labeling* (BGBL) algorithm proposed by Sadykov et al. (2021), which is capable of reducing the number of dominance checks, and thus the overall performance of the algorithm by using an advanced organization of labels in data structures called *buckets*.

The BGBL was designed for a more generic problem called the *resource constrained shortest path problem* (RCSP), in which one seeks

for a least-cost path between two given nodes of a graph such that accumulated consumption of resource (e.g., capacity, time) lie between a predefined interval for each arc of the path (Irnich and Desaulniers, 2005; Pugliese and Guerriero, 2013). More precisely, let $P = (v_0^P, v_1^P, v_2^P, \dots, v_{r-1}^P, v_r^P)$ be a path, and $a_\kappa^P = (v_{\kappa-1}^P, v_\kappa^P)$, $\kappa = 1, \dots, r$, be the κ-th arc of P. Each arc a consumes $\bar{q}_{a,\theta}$ of resource θ and has a predefined interval $[\bar{l}_{a,\theta}, \bar{u}_{a,\theta}]$ for the accumulated consumption of the referred resource. After traversing arc $a = a_\kappa^P$, the accumulated consumption for the resource θ, denoted as S_κ^θ, is defined by the following expression:

$$S_\kappa^\theta = \begin{cases} \max\{S_{\kappa-1}^\theta + \bar{q}_{a,\theta}, \bar{l}_{a,\theta}\} & \text{if } \kappa > 1 \\ \max\{\bar{q}_{a,\theta}, \bar{l}_{a,\theta}\} & \text{if } \kappa = 1 \end{cases} \tag{33}$$

Path P is feasible if, for all arcs $a = a_\kappa^P$, $\kappa = 1, \dots, r$, the value S_κ^θ lies in $[\bar{l}_{a,\theta}, \bar{u}_{a,\theta}]$ for all resources θ. Note that the resources are disposable, i.e., the resource can be dropped so that the lower bound $\bar{l}_{a,\theta}$ is satisfied. In our algorithm, there can be up to three resources depending on the variant: time resource, pickup resource (a capacity resource concerning only pickup demands), and a similar delivery resource. All of them have different values of consumption for each arc $a = (i, j) \in A_d$, $d = \{1, \dots, m\}$, and intervals for the accumulated consumption. Specifically, the consumption for the pickup, delivery, and time resources are defined as p_j, q_j, and t_a, respectively. The intervals are equal to $[0, Q^k]$ for the pickup and delivery resources, and to $[e_j, l_j]$ for the time resource. The intervals for the demand resources can be tightened, as further shown in Propositions 1 and 2.

A label L represents a partial path P containing the following information: $v(L)$, the last customer visited by P; $p(L)$, the accumulated consumption for the pickup resource; $q(L)$, the accumulated consumption for the delivery resource; $t(L)$, the accumulated consumption for the time resource; $\Pi(L) \subseteq V_C$, the set of forbidden extensions due to the *ng*-path relaxation constraints; $\bar{c}(L)$, the reduced cost of partial path P; and a vector S^L of states in the active R1Cs (see Algorithm 1). The following conditions are sufficient for a label L to dominate a label L':

(I) $v(L) = v(L')$
(II) $p(L) \leq p(L')$
(III) $q(L) \leq q(L')$
(IV) $\Pi(L) \subseteq \Pi(L')$
(V) $\bar{c}(L) < \bar{c}(L') + \sum_{\substack{l=1 \\ s_l^L > s_l^{L'}}}^{n^{\text{R1C}}} \pi^l$.

Each bucket stores all labels whose accumulated consumption of resources for a subset of resources, called *main resources*, lies between specific intervals. In the so-called *bucket graph*, each node represents a bucket and there exists a *bucket arc* from bucket b_1 to bucket b_2 iff it is possible to generate a label that must be stored in b_2 by extending a label stored in b_1. Sadykov et al. (2021) proposed a procedure capable of removing bucket arcs provided lower and upper bounds and reduced costs. Our algorithm executes such a procedure right after the convergence of the column generation algorithm with a view of speeding up the pricing algorithm.

We now state a simple proposition that can be useful for tightening the formulation.

Proposition 1. *Let $R = (v_0^R, v_1^R, v_2^R, \dots, v_{r-1}^R, v_r^R)$ be a feasible route, where v_0^R and v_r^R represent the same depot, and Q_R be the capacity of the vehicle associated with R. Then,* $\max\{p_{v_i^R} + p_{v_{i+1}^R}, q_{v_i^R} + q_{v_{i+1}^R}\} + \sum_{j=i+2}^{r} q_{v_j^R} \leq Q_R$, $i = 0, \dots, r-2$.

Proof. This follows directly from the feasibility of R, as it implies that $q_{v_i^R} + q_{v_{i+1}^R} + \sum_{j=i+2}^{r} q_{v_j^R} \leq Q_R$ and $p_{v_i^R} + p_{v_{i+1}^R} + \sum_{j=i+2}^{r} q_{v_j^R} \leq Q_R$, $i = 0, \dots, r-2$. □

In light of Proposition 1, one can set the interval for the accumulated consumption of the delivery resource to $[\max\{p_i + p_j, q_i + q_j\}, Q_k]$ instead of $[0, Q_k]$ for all arcs $a = (i, j) \in A_d$. Such a trick potentially eliminates infeasible paths during the labeling and ultimately avoids the addition of some lazy cuts, as illustrated in the following example.

Consider $n = 3$, $m = 1$, $V = \{V_C = \{1, 2, 3\} \cup V_D = \{4\} \cup V_D' = \{5\}\}$, $|K| = 1$, and $Q_1 = 10$. Consider also that $(q_1, q_2, q_3) = (5, 2, 3)$ and $(p_1, p_2, p_3) = (6, 2, 2)$. Moreover, let S_κ^q and S_κ^p, $\kappa = \{1, \dots, r\}$, be the cumulative resource consumption according to associated with the delivery and pickup resources, respectively. For simplicity, we assume that all customers are part of the same route and are visited in the following sequence $4 \to 1 \to 2 \to 3 \to 5$. Clearly, there are no capacity violations when considering the resources in an independent fashion:

$$\begin{aligned}
&S_1^q = \max\{q_1, 0\} = \max\{5, 0\} = 5 \leq Q_1 \implies S_1^q \in [0, Q_1], \\
&S_2^q = \max\{S_1^q + q_2, 0\} = \max\{5 + 2, 0\} = 7 \leq Q_1 \implies S_2^q \in [0, Q_1], \\
&S_3^q = \max\{S_2^q + q_3, 0\} = \max\{7 + 3, 0\} = 10 \leq Q_1 \implies S_3^q \in [0, Q_1], \\
&S_1^p = \max\{p_1, 0\} = \max\{6, 0\} = 6 \leq Q_1, \implies S_1^p \in [0, Q_1], \\
&S_2^p = \max\{S_1^p + p_2, 0\} = \max\{6 + 2, 0\} = 8 \leq Q_1, \implies S_2^p \in [0, Q_1], \\
&S_3^p = \max\{S_2^p + p_3, 0\} = \max\{8 + 2, 0\} = 10 \leq Q_1, \implies S_3^p \in [0, Q_1],
\end{aligned}$$

but there is a violation after visiting the first and second customers if they are considered simultaneously, as indicated by the following expressions:

$$\begin{aligned}
&D = q_1 + q_2 + q_3 = 5 + 2 + 3 = 10, \\
&(D - S_1^q) + S_1^p = (10 - 5) + 6 = 5 + 6 = 11 > Q_1 \implies (D - S_1^q) + S_1^p \notin [0, Q_1], \\
&(D - S_2^q) + S_2^p = (10 - 7) + 8 = 3 + 8 = 11 > Q_1 \implies (D - S_2^q) + S_2^p \notin [0, Q_1], \\
&(D - S_3^q) + S_3^p = (10 - 10) + 10 = 0 + 10 = 10 \leq Q_1, \implies (D - S_3^q) + S_3^p \in [0, Q_1].
\end{aligned}$$

By applying the trick to the arcs, and taking into account that the resources are disposable, we have that:

$$\begin{aligned}
&S_1^q = \max\{q_1, \max\{p_1, q_1\}\} = \max\{5, \max\{6, 5\}\} = 6 \leq Q_1, \\
&S_2^q = \max\{S_1^q + q_2, \max\{p_1 + p_2, q_1 + q_2\}\} = \max\{6 + 2, \max\{6 + 2, 5 + 2\}\} = 8 \leq Q_1, \\
&S_3^q = \max\{S_2^q + q_3, \max\{p_2 + p_3, q_2 + q_3\}\} = \max\{8 + 3, \max\{2 + 2, 2 + 3\}\} = 11 > Q_1.
\end{aligned}$$

It is possible to observe that the capacity violation already occurs when considering the delivery resource alone, and thus customer 3 could not be served after customers 1 and 2 have been visited in this order. Therefore, the referred route would be forbidden without the need of lazy cuts. This suggests that fewer cuts of this type are likely to be added in practice. Analogously, we can state a second proposition concerning the pickup resource.

Proposition 2. *Let $R = (v_0^R, v_1^R, v_2^R, \dots, v_{r-1}^R, v_r^R)$ be a feasible route, where v_0^R and v_r^R represent the same depot, and Q_R be the capacity of the vehicle associated with R. Then,* $\sum_{j=0}^{i-1} p_{v_j^R} + p_{v_i^R} + p_{v_{i+1}^R} \leq \min\{Q_R, Q_R - (q_{v_i^R} + q_{v_{i+1}^R}) + (p_{v_i^R} + p_{v_{i+1}^R})\}$, $i = 1, \dots, r-1$.

Proof. Once again, this follows directly from the feasibility of R, as it implies that $\sum_{j=0}^{(i+1)} p_{v_j^R} \leq Q_R$ and $\sum_{j=0}^{i-1} p_{v_j^R} \leq Q_R - (q_{v_i^R} + q_{v_{i+1}^R})$, $i = 1, \dots, r-1$. □

In accordance with Proposition 2, one can define the interval for the accumulated consumption of the pickup resource as $[0, \min\{Q_k, Q_k - (q_i + q_j) + (p_i + p_j)\}]$ instead of $[0, Q_k]$ for all arcs $a = (i, j) \in A_d$. This adjustment also has the potential to reduce the number of lazy cuts by eliminating infeasible paths, as indicated in the example described in the Supplementary Material, in which we also provide a more detailed description regarding the structure of the pricing subproblem.

5.3. Solving the master problem

The linear relaxation of formulation F2($\mathcal{M}$), denoted as LP-F2($\mathcal{M}$), is solved by a two-stage column generation (CG) procedure (see Desaulniers et al., 2006 for further details on CG). As long as paths with negative reduced cost can be found, one runs the BGBL algorithm in a heuristic fashion, where each bucket keeps only the label with the best reduced cost (first stage). When the heuristic approach fails, one switches to the second stage, in which the BGBL algorithm is executed in an exact fashion. In the first (second) stage, up to 30 (150) variables are added to the restricted master problem. Both stages make use of the stabilization algorithm proposed by Pessoa et al. (2018) to speed up CG convergence.

5.4. Augmenting ng-memories

In some cases, it is not obvious to define *a priori* a collection of memories $\mathcal{M}$ with a good compromise between average memory size and LB. To circumvent this issue, Roberti and Mingozzi (2014) proposed a dynamic approach to define the memories. The idea is to augment the memories iteratively based on the cycles of the fractional solutions found by the CG procedure. Note that a cycle on a customer j can be forbidden if one adds j to the memory of all internal nodes of the cycle. We adopt the memory augmentation algorithm proposed by Bulhões et al. (2018) to determine which cycles of the fractional solution found by the CG procedure will be forbidden.

5.5. Branching

We apply a two-phase strong branching approach similar to those developed by Røpke (2012) and Pecin et al. (2017b). In the first phase, we select 50 branching candidates, where each candidate is a branching expression. Up to 50% of those candidates come from the branching history based on their pseudo-costs (Achterberg, 2007), while the remaining candidates correspond to the most fractional expressions (closest to 0.5 first) in the current fractional solution. For each candidate selected in the first phase, we evaluate both child nodes by a re-optimization of the final linear program (LP) of the parent node with the additional branching constraint. Such re-optimization is a rough estimation of the nodes since no column generation is performed. The increases in the linear relaxation values are combined according to the product rule (Achterberg, 2007) so as to score the candidate. The best five candidates are selected for the second phase, in which we apply heuristic column generation in the evaluation of the child nodes. Finally, the best candidate, according to the product rule, is selected for branching. In the following, we list the branching expressions considered in this work:

- A single x_a^{kd} or y_d variable (for all variants);
- The number of routes performed by vehicles of type $k \in K$ departing from depot $d \in \{1, \dots, m\}$, i.e., $\sum_{j \in V_C} x^{kd}_{(n+d,j)}$ (for all variants except for VRPSPTW and FDPPTW).
- The assignment of a customer $j \in V_C$ for a pair of vehicle type $k \in K$ and depot $d \in \{1, \dots, m\}$, i.e., $\sum_{a \in \delta^-_d(j)} x_a^{kd}$ (only for HVRPSPD, LRPSPD and MDVRPMPD).

6. Computational experiments

Our BCP algorithm was coded in C++ (g++, version 9.4.0) over the BapCod platform (Vanderbeck et al., 2017) with a VRPSolver extension (Pessoa et al., 2020). Computational experiments were executed on a single thread of an Intel® Xeon™ CPU E5-2650 v4 with 2.20 GHz and 128 GB of RAM running Linux Ubuntu 16.04.6. CPLEX 12.10 was used as an LP solver. A time limit of 12 h was imposed for the BCP algorithm, except when this duration was insufficient to solve the root node. Moreover, the algorithm was executed with an extended 48-hour time limit to confirm the optimality of instances nearly solved within 12 h.

The results obtained on each of the ten VRPSPD variants are summarized in Tables 5–14. Columns *Set*, *n*, and *#inst* respectively indicate the benchmark dataset, the number of customers, and the number of instances with that particular size. Column *#New opt* corresponds to the number of open instances that were solved to optimality, and column *#Improved LBs* specifies how many lower bounds were improved, including the number of instances for which dual bounds were provided for the first time, disregarding the proven optima. Column *#Known opt* represents the number of known optima obtained by other methods in the literature, among which we report those that have been *found* or *lost* by our BCP. Finally, columns *#Worse LBs* and *#No bounds* indicate the number of cases in which the best-known dual bounds were not improved or no dual bound was found, respectively. Furthermore, detailed results for each instance are provided in the Supplementary Material, including indications of the cases in which the relaxation of one of the resources was required to obtain valid LBs.

Regarding the classical VRPSPD, we tested the proposed algorithm on the SN and MG benchmarks, and the results are summarized in Table 5. BCP was capable of achieving important results on the former. More specifically, the two 75-customer instances that were open for almost 25 years were finally solved to optimality, and improved LBs were achieved for instances with 150 and 199 customers. For the latter benchmark, BCP managed to improve the LBs of 4 instances containing 200 customers and also to provide valid and relatively tight LBs for the 400-customer instances for the first time. On the other hand, our BCP struggled to find competitive LBs for the two 120-customer instances containing only four vehicles. This is not surprising as it is known that instances with a relatively large number of customers per route tend to favor BC algorithms.

For the VRPMPD, we considered the SN benchmark, and a summary of the results is compiled in Table 6. Four LBs were improved for the larger instances, and two long-standing open instances with 75 customers were solved to optimality.

To our knowledge, there are no LBs available for the benchmark instances associated with the VRPSPDTL. Therefore, we are now providing the first valid LBs for the 14 instances of the SN set, as summarized in Table 7. The route duration constraint makes the problem more challenging than its classical counterpart, requiring the inclusion of an additional resource to keep track of the total spent on a route. Nonetheless, it is worth highlighting that optimal solutions were found for all 50-customer instances, as well as for two instances with 100 customers.

Likewise, the same observation applies to the VRPMPDTL, in which the results are outlined in Table 8. In this case, optimal solutions were obtained for 100%, 66.67%, and 50% of the instances containing 50, 75, and 100 customers, respectively. In addition, except for a few cases, the LBs provided are relatively tight.

Table 9 summarizes the results obtained for the VRPSPDTW, more specifically, those attained on the W12 benchmark. It can be observed that the LBs of all instances were either equaled or improved by the proposed algorithm. In particular, our BCP was capable of finding the optimal solutions for all instances with up to 50 customers. Moreover, 30 instances containing 100 customers had their optima proven for the first time, whereas the LBs of the other 20 were improved.

The results achieved on the FDPPTW, considering the W13 benchmark, are very similar to those obtained on the VRPSPDTW, as can be seen in Table 10. In this case, the values of almost all LBs achieved by BCP were greater than or equal to be best known. Except for one case, all instances with up to 50 customers were solved to optimality. Six new optima were also found for the 100-customer instances. Nevertheless, BCP failed to obtain valid LBs for five instances of this size, as indicated in the *#No bounds* column. In these particular cases, the pricing subproblem resolution was interrupted due to the time limit associated with the labeling algorithm, even when considering the relaxation of

Table 5
Summary of results obtained for the VRPSPD instances.

Set	*n*	#*inst*	#New opt	#Improved LBs	#Known opt found	#Known opt lost	#Worse LBs
SN	50	2	0	0	2	0	0
	75	2	2	0	0	0	0
	100	4	0	0	0	2	2
	120	2	0	0	0	0	2
	150	2	0	1	0	0	1
	199	2	0	2	0	0	0
MG	100	6	0	0	6	0	0
	200	6	0	4	0	2	0
	400	6	0	6	0	0	0

Table 6
Summary of results obtained for the VRPMPD instances.

Set	*n*	#*inst*	#New opt	#Improved LBs	#Known opt found	#Known opt lost	#Worse LBs
SN	50	3	0	0	3	0	0
	75	3	2	0	1	0	0
	100	6	0	0	4	2	0
	120	3	0	0	0	1	2
	150	3	0	1	0	0	2
	199	3	0	3	0	0	0

Table 7
Summary of results obtained for the VRPSPDTL instances.

Set	*n*	#*inst*	#New **opt**	#Improved LBs	#Known opt found	#Known opt lost	#Worse LBs
SN	50	2	2	0	–	–	–
	75	2	0	2	–	–	–
	100	4	2	2	–	–	–
	120	2	0	2	–	–	–
	150	2	0	2	–	–	–
	199	2	0	2	–	–	–

Table 8
Summary of results obtained for the VRPMPDTL instances.

Set	*n*	#*inst*	#New opt	#Improved LBs	#Known opt found	#Known opt lost	#Worse LBs
SN	50	3	3	0	–	–	–
	75	3	2	1	–	–	–
	100	6	3	3	–	–	–
	120	3	0	3	–	–	–
	150	3	0	3	–	–	–
	199	3	0	3	–	–	–

Table 9
Summary of results obtained for the VRPSPDTW instances.

Set	*n*	#*inst*	#New opt	#Improved LBs	#Known opt found	#Known opt lost	#Worse LBs
W12	10	3	0	0	3	0	0
	25	3	2	0	1	0	0
	50	3	2	0	1	0	0
	100	56	30	20	6	0	0

Table 10
Summary of results obtained for the FDPPTW instances.

Set	*n*	#*inst*	#New opt	#Improved LBs	#Known opt found	#Known opt lost	#Worse LBs	#No bounds
W13	5	3	0	0	3	0	0	0
	10	3	2	0	1	0	0	0
	25	3	3	0	0	0	0	0
	50	3	2	1	0	0	0	0
	100	56	6	43	0	0	0	7

Table 11
Summary of results obtained for the AVRPSPD instances.

Set	*n*	*#inst*	#New opt	#Improved LBs	#Known opt found	#Known opt lost	#Worse LBs
RZC6	50	40	20	0	20	0	0
RZC7	50	2	0	0	2	0	0

Table 12
Summary of results obtained for the HVRPSPD instances.

Set	*n*	*#inst*	#New opt	#Improved LBs	#Known opt found	#Known opt lost	#Worse LBs
Avic1	10	2	0	0	2	0	0
	15	2	1	0	1	0	0
	20	2	2	0	0	0	0
	35	2	2	0	0	0	0
	50	2	1	1	0	0	0
	75	2	0	2	0	0	0
	100	2	0	2	0	0	0

Table 13
Summary of results obtained for the MDVRPMPD instances.

Set	*n*	*#inst*	#New opt	#Improved LBs	#Known opt found	#Known opt lost	#Worse LBs
SN	50	6	3	0	3	0	0
	75	3	3	0	0	0	0
	100	12	10	0	1	0	1
	249	12	0	12	0	0	0

one of the resources. Moreover, two instances were disregarded since they shared the same input data as their corresponding VRPSPDTW instances. This exclusion is also reflected in the same column.

For what concerns the AVRPSPD, we have run experiments on the RZC6 and RZC7 benchmarks. The LBs of all 42 instances were either equaled or improved, and new optimal values were attained for 20 instances, as presented in Table 11.

Table 12 compiles the results obtained on the Avc1 benchmark associated with the HVRPSPD. Since we are not aware of any exact approach for this problem, we believe to have provided the first valid LBs for all 14 instances of the referred dataset, of which 9 of them were proven to be optimal. In addition, we were capable of improving the UBs of three instances, as reported in the Supplementary Material.

The results achieved for the MDVRPMPD, taking into account the SN benchmark, are outlined in Table 13. We can verify that 20 instances were solved to optimality (16 of them for the first time). Furthermore, we provide the first valid LBs for the 249-customer instances.

With respect to the LRPSPD, we have considered three benchmarks: Bar11, Prod11, and Prod12, and the results are compiled in Table 14. From the third instance set, we omitted instances with 20 customers since they are already included in the second instance set. Regarding Bar11, our BCP could solve all 50-customer instances to optimality, as well as two instances with 55 customers, all of them for the first time. Also, the LBs of half of the instances with 55, 75, 85, and 88 customers were improved. Concerning Prod11, the proposed algorithm managed to prove the optimality of 20 instances with 50 customers for the first time, and it was also capable of improving the LBs of many 100-customer instances. As for Prod12, our method obtained a total of 28 optimal solutions for open instances. Moreover, it is worth mentioning that we have identified inconsistencies in the objective values of the optimal solutions for a few instances of Prod11. In four cases, we have found optimal values greater than those reported in Karaoglan et al. (2011), and in two cases, our BCP found optimal values smaller than those presented in the referred work. We then implemented the model described in Karaoglan et al. (2011) and solved it using CPLEX. The results achieved by the solver matched those found by our BCP, thus confirming the consistency of our results.

Finally, Table 15 presents the overall results for all problems considered in this work. The proposed BCP algorithm was capable of finding the optimal solutions of 44.3% open problems, as well as of achieving 92.6% of the known optima, and of improving the LBs of 69.9% of the instances that remained open. We believe that such figures ratify the relevance of the method when it comes to systematically obtaining strong dual bounds for a broad class of VRPSPDs.

7. Concluding remarks

This work presented a BCP algorithm for solving ten existing VRPSPD variants with different attributes that can be seen as particular cases of a generic problem, here denoted as HLRPSPDTW. The BCP approach was implemented using the VRPSolver framework, considering pickup and delivery demands as independent resources in the underlying proposed mathematical formulation. Lazy cuts are employed to disregard solutions in which the sum of delivery and pickup demands are violated in the middle of the route.

We tested our BCP method on 538 benchmark instances, for which the algorithm was capable of improving the lower bounds of 312 open problems, with 166 of them proven to be optimal. Moreover, 151 known optima were also found by our exact procedure. BCP was outperformed in only 12.6% of the instances, thus suggesting a very consistent performance despite its broad generality.

As for future work, one could devise an alternative pricing algorithm with the ability to efficiently address the interdependence between pickup and delivery demands, thus avoiding the use of lazy cuts. Extensions of the algorithm to cope with other variants of the problem considering uncertainties (e.g., robustness or stochastic resources) and environmental aspects (e.g., electric vehicles) could also be worth investigating.

Table 14
Summary of results obtained for the LRPSPD instances.

Set	*n*	*#inst*	#New opt	#Improved LBs	#Known opt found	#Known opt lost	#Worse LBs
Bar11	8	4	0	0	4	0	0
	12	4	0	0	4	0	0
	21	4	0	0	4	0	0
	22	4	0	0	4	0	0
	27	4	0	0	4	0	0
	29	4	0	0	4	0	0
	32	8	0	0	8	0	0
	36	4	0	0	4	0	0
	50	4	4	0	0	0	0
	55	4	2	2	0	0	0
	75	4	0	2	0	0	2
	85	4	0	2	0	0	2
	88	4	0	2	0	1	1
	100	4	0	0	0	0	4
Prod11	20	16	4	0	12	0	0
	50	24	20	1	0	1	2
	100	48	3	16	0	0	29
Prod12	10	16	0	0	16	0	0
	15	16	1	0	15	0	0
	25	24	11	0	8	3	2
	30	24	16	0	4	0	4

Table 15
Summary of the results obtained for all the problems.

Problem	Set	*#inst*	#New opt	#Improved LBs	#Known opt found	#Known opt lost	#Worse LBs	#No bounds
VRPSPD	SN	14	2	3	2	2	5	0
	MG	18	0	10	6	2	0	0
VRPMPD	SN	21	2	4	8	3	4	0
VRPSPDTL	SN	14	4	10	–	–	–	0
VRPMPDTL	SN	21	8	13	–	–	–	0
VRPSPDTW	W12	65	34	20	11	0	0	0
FDPPTW	W13	68	13	44	4	0	0	7
AVRPSPD	RZC6	40	20	0	20	0	0	0
	RZC7	2	0	0	2	0	0	0
HVRPSPD	Avci1	14	6	5	3	0	0	0
MDVRPMPD	SN	33	16	12	4	0	1	0
LRPSPD	Bar11	60	6	8	36	1	9	0
	Prod11	88	27	17	12	1	31	0
	Prod12	80	28	0	43	3	6	0
Total		538	166	146	151	12	56	7

CRediT authorship contribution statement

Rafael Praxedes: Methodology, Software, Validation, Formal analysis, Investigation, Writing – original draft, Writing – review & editing. **Teobaldo Bulhões:** Methodology, Validation, Formal analysis, Investigation, Resources, Writing – review & editing, Supervision, Funding acquisition. **Anand Subramanian:** Conceptualization, Methodology, Validation, Formal analysis, Investigation, Resources, Writing – original draft, Writing – review & editing, Supervision, Funding acquisition. **Eduardo Uchoa:** Methodology, Validation, Formal analysis, Writing – review & editing.

Data availability

Data will be made available on request.

Acknowledgments

This research was partially supported by: (i) CNPq, grants 309580/2021-8, 406245/2021-5, 314088/2021-0, and 313601/2018-6; (ii) CAPES, under Finance Code 001; (iii) CAPES PrInt UFF, grant 88881; (iv) Paraíba State Research Foundation (FAPESQ), grants 261/2020, 2021/3182, and 041/2023; (v) Universidade Federal da Paraíba through the Public Call n. 03/2020 "Produtividade em Pesquisa PROPESQ/PRPG/UFPB", proposal codes PVL13395-2020 and PVL13400-2020; and (vi) FAPERJ, grant E-26/202.887/2017.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.cor.2023.106467.

References

Achterberg, T., 2007. Constraint Integer Programming (Ph.D. thesis). Technische Universitat Berlin.

Agarwal, Y.K., Venkateshan, P., 2020. A new model for the asymmetric vehicle routing problem with simultaneous pickup and deliveries. Oper. Res. Lett. 48, 48–54.

Agarwal, Y.K., Venkateshan, P., 2021. New valid inequalities for the symmetric vehicle routing problem with simultaneous pickup and deliveries. Networks 79 (4), 537–556.

Agius, M., Absi, N., Feillet, D., Garaix, T., 2022. A branch-and-price algorithm for a routing problem with inbound and outbound requests. Comput. Oper. Res. 146, 105896.

Angelelli, E., Mansini, R., 2002. The vehicle routing problem with time windows and simultaneous pick-up and delivery. In: Klose, A., Speranza, M.G., Van Wassenhove, L.N. (Eds.), Quantitative Approaches To Distribution Logistics and Supply Chain Management. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 249–267.

Avci, M., Topaloglu, S., 2016. A hybrid metaheuristic algorithm for heterogeneous vehicle routing problem with simultaneous pickup and delivery. Expert Syst. Appl. 53, 160–171.

Baldacci, R., Mingozzi, A., Roberti, R., 2011. New route relaxation and pricing strategies for the vehicle routing problem. Oper. Res. 59 (5), 1269–1283.
Barreto, S., Ferreira, C., Paixão, J., Santos, B.S., 2007. Using clustering analysis in a capacitated location-routing problem. European J. Oper. Res. 179, 968–977.
Battarra, M., Cordeau, J.F., Iori, M., 2014. Chapter 6: Pickup-and-delivery problems for goods transportation. In: Vehicle Routing. SIAM, pp. 161–191.
Berbeglia, G., Cordeau, J.F., Gribkovskaia, I., Laporte, G., 2007. Static pickup and delivery problems: a classification scheme and survey. TOP 15, 1–31.
Berbeglia, G., Cordeau, J.F., Laporte, G., 2010. Dynamic pickup and delivery problems. European J. Oper. Res. 202, 8–15.
Bouanane, K., Amrani, M.E., Benadada, Y., 2022. The vehicle routing problem with simultaneous delivery and pickup: a taxonomic survey. Int. J. Logist. Syst. Manage. 41 (1–2), 77–119.
Bulhões, T., Sadykov, R., Uchoa, E., 2018. A branch-and-price algorithm for the Minimum Latency Problem. Comput. Oper. Res. 93, 66–78.
Caprara, A., Fischetti, M., 1996. 0, 1/2-Chvátal-Gomory cuts. Math. Programm. Ser. B 74 (3), 221–235.
Chen, J.F., Wu, T.H., 2006. Vehicle routing problem with simultaneous deliveries and pickups. J. Oper. Res. Soc. 57 (5), 579–587.
Contardo, C., Martinelli, R., 2014. A new exact algorithm for the multi-depot vehicle routing problem under capacity and route length constraints. Discrete Optim. 12, 129–146.
Dell'Amico, M., Righini, G., Salani, M., 2006. A branch-and-price approach to the vehicle routing problem with simultaneous distribution and collection. Transp. Sci. 40 (2), 235–247.
Desaulniers, G., Desrosiers, J., Solomon, M.M., 2006. Column Generation, Vol. 5. Springer Science & Business Media.
Dethloff, J., 2001. Vehicle routing and reverse logistics: the vehicle routing problem with simultaneous delivery and pick-up. OR-Spektrum 23 (1), 79–96.
Gehring, H., Homberger, J., 2002. Parallelization of a two-phase metaheuristic for routing problems with time windows. J. Heuristics 8 (3), 251–276.
Irnich, S., Desaulniers, G., 2005. Shortest path problems with resource constraints. In: Column Generation. Springer US, Boston, MA, pp. 33–65.
Jepsen, M., Petersen, B., Spoorendonk, S., Pisinger, D., 2008. Subset-Row inequalities applied to the vehicle-routing problem with time windows. Oper. Res. 56 (2), 497–511.
Karaoglan, I., Altiparmak, F., Kara, I., Dengiz, B., 2011. A branch and cut algorithm for the location-routing problem with simultaneous pickup and delivery. European J. Oper. Res. 211, 318–332.
Karaoglan, I., Altiparmak, F., Kara, I., Dengiz, B., 2012. The location-routing problem with simultaneous pickup and delivery: Formulations and a heuristic approach. Omega 40, 465–477.
Koç, Ç., Laporte, G., Tükenmez, İ., 2020. A review of vehicle routing with simultaneous pickup and delivery. Comput. Oper. Res. 122.
Laporte, G., Nobert, Y., 1983. A branch and bound algorithm for the capacitated vehicle routing problem. Oper. -Res. -Spektrum 5 (2), 77–85.
Lysgaard, J., Letchford, A.N., Eglese, R.W., 2004. A new branch-and-cut algorithm for the capacitated vehicle routing problem. Math. Program. 100.
Min, H., 1989. The multiple vehicle routing problem with simultaneous delivery and pick-up points. Transp. Res. A General 23 (5), 377–386.
Mitra, S., 2005. An algorithm for the generalized vehicle routing problem with backhauling. Asia-Pac. J. Oper. Res. 22 (02), 153–169.
Montané, F.A.T., Galvão, R.D., 2006. A Tabu search algorithm for the vehicle routing problem with simultaneous pick-up and delivery service. Comput. Oper. Res. 33, 595–619.
Parragh, S.N., Doerner, K.F., Hartl, R.F., 2008a. A survey on pickup and delivery problems. J. Betriebswirtschaft 58, 21–51.
Parragh, S.N., Doerner, K.F., Hartl, R.F., 2008b. A survey on pickup and delivery problems: Part II: Transportation between pickup and delivery locations. J. Betriebswirtschaft 58, 81–117.
Pecin, D., Contardo, C., Desaulniers, G., Uchoa, E., 2017a. New enhancements for the exact solution of the vehicle routing problem with time windows. INFORMS J. Comput. 29 (3), 489–502.
Pecin, D., Pessoa, A., Poggi, M., Uchoa, E., 2017b. Improved branch-cut-and-price for capacitated vehicle routing. Math. Program. Comput. 9, 61–100.
Pecin, D., Pessoa, A., Poggi, M., Uchoa, E., Santos, H., 2017c. Limited memory rank-1 cuts for vehicle routing problems. Oper. Res. Lett. 45 (3), 206–209.
Pessoa, A., Sadykov, R., Uchoa, E., Vanderbeck, F., 2018. Automation and combination of linear-programming based stabilization techniques in column generation. INFORMS J. Comput. 30 (2), 339–360.
Pessoa, A., Sadykov, R., Uchoa, E., Vanderbeck, F., 2020. A generic exact solver for vehicle routing and related problems. Math. Program. 183, 483–523.
Poggi, M., Uchoa, E., 2003. Integer program reformulation for robust branch-and-cut-and-price. In: Wolsey, L. (Ed.), Annals of Mathematical Programming in Rio. Búzios, Brazil, pp. 56–61.
Prins, C., Prodhon, C., Calvo, R., 2004. Nouveaux algorithmes pour le problème de localisation et routage sous contraintes de capacité. MOSIM'04 2, 1115–1122.
Pugliese, L.D.P., Guerriero, F., 2013. A survey of resource constrained shortest path problems: Exact solution approaches. Networks 62 (3), 183–200.
Qu, Y., Bard, J.F., 2015. A branch-and-price-and-cut algorithm for heterogeneous pickup and delivery problems with configurable vehicle capacity. Transp. Sci. 49, 254–270.
Rieck, J., Zimmermann, J., 2013. Exact solutions to the symmetric and asymmetric vehicle routing problem with simultaneous delivery and pick-up. Bus. Res. 6, 77–92.
Roberti, R., Mingozzi, A., 2014. Dynamic ng-Path relaxation for the delivery man problem. Transp. Sci. 48 (3), 413–424.
Røpke, S., 2012. Branching decisions in branch-and-cut-and-price algorithms for vehicle routing problems. Presentation In Column Generation 2012.
Sadykov, R., Uchoa, E., Pessoa, A., 2021. A bucket graph–Based labeling algorithm with application to vehicle routing. Transp. Sci. 55 (1), 4–28.
Salhi, S., Nagy, G., 1999. A cluster insertion heuristic for single and multiple depot vehicle routing problems with backhauling. J. Oper. Res. Soc. 50 (10), 1034–1042.
Solomon, M.M., 1987. Algorithms for the vehicle routing and scheduling problems with time window constraints. Oper. Res. 35 (2), 254–265.
Subramanian, A., 2012. Heuristic, Exact and Hybrid Approaches for Vehicle Routing Problems (Ph.D. thesis). Universidade Federal Fluminense, Niterói, RJ.
Subramanian, A., Uchoa, E., Ochi, L.S., 2010. New lower bounds for the vehicle routing problem with simultaneous pickup and delivery. In: Festa, P. (Ed.), Experimental Algorithms. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 276–287.
Subramanian, A., Uchoa, E., Pessoa, A.A., Ochi, L.S., 2011. Branch-and-cut with lazy separation for the vehicle routing problem with simultaneous pickup and delivery. Oper. Res. Lett. 39, 338–341.
Subramanian, A., Uchoa, E., Pessoa, A.A., Ochi, L.S., 2013. Branch-cut-and-price for the vehicle routing problem with simultaneous pickup and delivery. Optim. Lett. 7, 1569–1581.
Toth, P., Vigo, D., 1997. An exact algorithm for the vehicle routing problem with backhauls. Transp. Sci. 31 (4), 372–385.
Vanderbeck, F., Sadykov, R., Tahiri, I., 2017. BaPCod — a generic branch-and-price code. URL: https://realopt.bordeaux.inria.fr/?page_id=2.
Wang, H.F., Chen, Y.Y., 2012. A genetic algorithm for the simultaneous delivery and pickup problems with time window. Comput. Ind. Eng. 62, 84–95.
Wang, H.F., Chen, Y.Y., 2013. A coevolutionary algorithm for the flexible delivery and pickup problem with time windows. Int. J. Prod. Econ. 141, 4–13.